53-ai-coding-agents

Module 53: AI Coding Agents & Workflow

Level: Advanced
Prerequisites: Module 1 (AI Fundamentals), Module 38 (AI Coding Tools Pengenalan)
Estimated Time: 4 sessions × 2–3 hours

Description

Module ini adalah **deep dive** ke AI Coding Agents — lanjutan dari Module 38 yang masih pengenalan. Fokus pada praktik langsung menggunakan multi-agent workflow, AI coding loop, dan penguasaan Hermes Agent sebagai orchestrator.

Setelah module ini, kamu akan bisa: - Membandingkan 5+ AI coding tools dan memilih yang tepat per skenario - Membangun multi-agent workflow dengan orchestrator + worker agents - Menjalankan AI Coding Loop (plan → code → test → review → refactor) secara disiplin - Menguasai Hermes Agent: delegation, skills, kron, MCP, kanban

Output Target

By end of module, kamu punya: - **Workflow AI Agent yang efektif** — dokumentasi comparison + decision tree kapan pakai tool mana - **Orchestrator script** yang manage multi-agent task - **3+ loop iteration** pada satu fitur (plan → code → test → review → refactor) - **Hermes Agent setup** dengan custom skill, profile, dan kanban board

Sessions

Figure 1: Banner

Session	Topic	Description
01	AI Coding Tools Comparison	Bandingkan Claude Code, Codex, Cursor, Copilot, Hermes Agent — kapan pake yang mana
02	Multi-Agent Workflow	Orchestrator + worker, role-based agents, parallel vs sequential execution
03	AI Coding Loop	Plan → Code → Test → Review → Refactor, loop efficiency
04	Hermes Agent Mastery	Delegation, skills, kron, profiles, MCP, kanban — jadi power user

Tools Used

- Claude Code CLI
- Codex CLI (OpenAI)
- Cursor IDE
- GitHub Copilot
- Hermes Agent (utama)
- Python 3.11+
- Git

Assessment

- Latihan praktik setiap sesi (wajib dikerjakan)
- Final project: setup Hermes Agent workflow lengkap + dokumentasi comparison

Session 01: AI Coding Tools Comparison

Level: Advanced
Duration: 2–3 hours
Objective: Pahami perbedaan, kelebihan, dan kekurangan 5 AI coding tools utama. Tentukan kapan pakai yang mana.

---

1. Overview Tools

Ada 5 AI coding tools yang dominan di 2025–2026. Masing-masing punya pendekatan berbeda:

Tool	Pendekatan	Interface Utama	Ekosistem
Claude Code	CLI-native agent	Terminal	Anthropic, MCP
Codex (OpenAI)	CLI-native + sandbox	Terminal	OpenAI, sandbox
Cursor	IDE-integrated	Editor GUI	VS Code fork
GitHub Copilot	IDE-plugin	Editor inline	GitHub, VS Code/JetBrains
Hermes Agent	Agent orchestrator	Terminal + tools	Nous Research, MCP

---

2. Claude Code

Cara Kerja

- **claude CLI** — terminal-based agent yang bisa membaca/menulis file, execute command, manage git
- **ACP Protocol** (Agent Communication Protocol) — standard untuk agent-to-agent communication
- **Thinking Mode** — reasoning depth configurable, untuk task kompleks
- **Edit Mode** — two-step: plan dulu, baru execute

Kelebihan

- Context window besar (200K tokens)
- ACP protocol memungkinkan multi-agent
- Edit mode prevent premature execution
- Bagus untuk refactoring large codebase

Kekurangan

- CLI-only, no GUI
- Dependency pada Anthropic API
- Cost per token relatif tinggi
- Learning curve untuk ACP

Use Case

- Codebase besar >10K file — perlu context besar
- Refactoring kompleks

- Multi-agent orchestration via ACP

---

3. Codex (OpenAI)

Cara Kerja

- **codex CLI** — terminal-based, fork dari philosophy yang sama dengan Claude Code
- **Sandbox** — setiap eksekusi diisolasi, rollback otomatis kalau error
- **Automatic Test** — generate test langsung dari spec
- Integrasi dengan OpenAI API (o-series models)

Kelebihan

- Sandbox aman untuk eksperimen
- Automatic test generation built-in
- Cost lebih murah dari Claude Code
- o-series models bagus untuk coding

Kekurangan

- Sandbox kadang terlalu restriktif
- Context window lebih kecil (128K)
- Community tools masih kurang
- Masih relatif baru (2025)

Use Case

- Project baru — sandbox proteksi
- Test-heavy development
- Budget terbatas

---

4. Cursor

Cara Kerja

- **Rules** — custom instructions yang selalu aktif (.cursorrules)
- **Inline Editing** — edit langsung di file dengan Cmd+K
- **Chat + Composer** — sidebar chat + multi-file edit composer
- **Agent Mode** — agent yang bisa read/write/run command otomatis

Kelebihan

- GUI paling mature — visual diff, inline suggestions
- Rules system powerful untuk consistency
- Multi-file editing via Composer
- Context-aware — baca seluruh project

Kekurangan

- Berat di RAM/CPU (Electron app)
- Dependency pada VS Code ecosystem
- Agent mode kadang terlalu agresif
- Subscription-based

Use Case

- Day-to-day development — GUI productivity
- Project dengan coding standards ketat
- Team dengan shared rules

---

5. GitHub Copilot

Cara Kerja

- **Copilot Chat** — AI chat inside IDE
- **Copilot Edits** — multi-file edit mode
- **Inline Completion** — tab completion real-time
- Integrasi mendalam VS Code, JetBrains, Neovim

Kelebihan

- Inline completion paling cepat dan akurat
- Integrasi GitHub — PR review, issues
- Multi-IDE support luas
- Copilot Edits powerful untuk refactor

Kekurangan

- Agent mode tidak sekuat Cursor/Claude Code
- Context terbatas pada file yang terbuka
- Kadang suggest nonsense code
- Perlu internet terus-menerus

Use Case

- Daily coding — boilerplate, completion
- GitHub-heavy workflow (PR, CI/CD)
- Multi-IDE team

---

6. Hermes Agent

Cara Kerja

- **Delegation** — agent utama delegasi task ke sub-agent (single + batch)
- **Skills** — reusable workflow yang bisa di-create, di-version, di-improve
- **Kron** — cron job scheduling untuk periodic tasks
- **Kanban** — board management multi-session
- **MCP** (Model Context Protocol) — add custom tools, webhook triggers

Kelebihan

- Full control — open source
- Multi-agent orchestration built-in
- Skills system untuk workflow reuse
- Kanban untuk project management
- MCP untuk extensibility

Kekurangan

- Setup lebih kompleks
- CLI-first, belum ada GUI matang
- Community lebih kecil
- Documentation masih bertumbuh

Use Case

- Complex workflow orchestration
- Multi-agent systems
- Production automation
- Power users who want full control

---

7. Comparison Matrix

Kriteria	Claude Code	Codex	Cursor	Copilot	Hermes Agent
Setup complexity	Medium	Medium	Low	Low	Medium-High
Context window	200K	128K	~varies	~small	~varies
Cost (est monthly)	$20-100	$10-50	$20	$10 (business)	Free (OSS)
GUI	❌	❌	✅	✅	❌ (CLI)
Multi-agent	✅ (ACP)	❌	❌	❌	✅ built-in
Sandbox	✅	✅	❌	❌	✅ (terminal)
Skills/Reusable	✅	❌	Rules	❌	✅ Skills
Kanban/PM	❌	❌	❌	❌	❌
Offline mode	❌	❌	⚠ (hybrid)	❌	✅ (local models)
Open source	❌	✅	❌	❌	✅

Decision Tree: Kapan Pakai Yang Mana

```
Task Complexity?
├── Simple completion / boilerplate → Copilot
├── Multi-file edit, daily dev → Cursor
├── Large codebase refactor → Claude Code
├── New project + test-heavy → Codex
└── Multi-agent workflow / automation → Hermes Agent
```

---

8. Cost Analysis

Tool	Model	Approx Cost/1K tokens	Daily typical usage	Monthly est
Claude Code	Claude Opus 4	$0.015 / $0.075	50K-200K tokens	$20-100
Codex	o3/o4-mini	$0.010 / $0.040	100K-300K tokens	$10-50

Tool	Model	Approx Cost/1K tokens	Daily typical usage	Monthly est
Cursor	Multi-model	Flat $20/mo	Unlimited	$20
Copilot	Multi-model	Flat $10/mo	2K completions/mo	$10
Hermes Agent	Any (user choice)	API cost only	Varies	$0-50

> **Note:** Hermes Agent cost tergantung model yang dipake. Bisa pake local LLM (ollama) = $0 API cost.

---

9. Latihan

Latihan 1: Setup 2 Tools Berbeda

1. Pilih 2 tools dari 5 di atas (rekomendasi: Hermes Agent + 1 tool lain)
2. Setup kedua tool di environment kamu
3. Kerjakan task yang SAMA dengan kedua tool:

```
# Task: Buat REST API endpoint untuk user registration
# - POST /api/register
# - Input: username, email, password
# - Output: user object (tanpa password), JWT token
# - Validation: email format, password min 8 chars
# - Error handling: duplicate username/email
```

4. Catat perbedaan:
   - Waktu yang dibutuhkan
   - Kualitas output
   - Jumlah prompt/iteration
   - Error rate
   - Code style

Latihan 2: Comparison Report

Buat dokumentasi comparison dalam format:

```
## Tool A vs Tool B

### Setup Experience
[how easy was setup?]
```

```
### Task Performance
[which one faster? which one better quality?]

### Code Quality
[style, best practices, error handling]

### Recommendation
[kapan pakai A, kapan pakai B]
```

Delivery

- Simpan comparison report di `53-ai-coding-agents/labs/comparison-report.md`
- Siap presentasi 5 menit per tool

Key Takeaways

1. **Tidak ada satu tool terbaik** — setiap tool punya niche
2. **Copilot** = daily driver untuk completion cepat
3. **Cursor** = GUI productivity untuk development
4. **Claude Code** = complex codebase refactoring
5. **Codex** = test-heavy + new project with sandbox
6. **Hermes Agent** = orchestrator + automation power user
7. **Combine tools** — gunakan multiple tools untuk different tasks

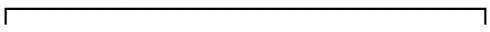
Session 02: Multi-Agent Workflow

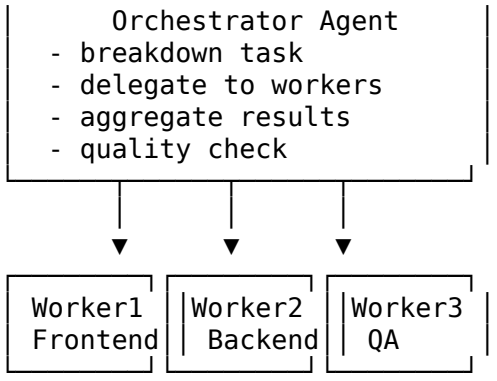
> **Level:** Advanced
> **Duration:** 2–3 hours
> **Objective:** Bangun multi-agent workflow dengan orchestrator + worker pattern. Pahami role-based agents, tools delegation, context isolation.

---

1. Orchestrator + Worker Pattern

Pattern paling fundamental di multi-agent systems. Satu **orchestrator agent** manage beberapa **worker agents** yang masing-masing handle subtask spesifik.





Orchestrator Responsibilities

Task	Description
Task breakdown	Pecah task besar jadi sub-task kecil yang independen
Delegation	Assign sub-task ke worker agent yang tepat
Context passing	Berikan context yang relevan ke setiap worker
Parallel/sequential	Tentukan mana yang bisa parallel, mana sequential
Result aggregation	Gabung hasil dari semua worker
Quality check	Validasi hasil sebelum final

Worker Responsibilities

Task	Description
Execution	Kerjakan sub-task sesuai instruksi
Reporting	Laporkan hasil + status
Error handling	Tangani error lokal, report ke orchestrator
Context isolation	Tidak perlu tahu task worker lain

---

2. Role-Based Agents

Assign roles ke workers berdasarkan expertise:

Frontend Agent

```
frontend_agent_prompt = """You are a Frontend Development Agent.
Your expertise:
- React/Next.js, Tailwind CSS, TypeScript
- Responsive design, accessibility
- Component architecture
- State management

Task: {task}
Context: {context}
Output: Complete frontend code with:
- Component files
- Styles
- Tests (if applicable)
- Usage example
"""
```

Backend Agent

```
backend_agent_prompt = """You are a Backend Development Agent.
Your expertise:
- Python/FastAPI, Node.js/NestJS
- Database design, API design
- Authentication, authorization
- Performance optimization

Task: {task}
Context: {context}
Output: Complete backend code with:
- API endpoints
- Database models
- Business logic
- Tests
- Documentation
"""
```

QA Agent

```
qa_agent_prompt = """You are a QA/Testing Agent.
Your expertise:
- Unit testing, integration testing, E2E testing
- Test coverage analysis
- Bug detection and reporting
- Performance testing
```

```
Task: {task}
Context: {context}
Output: Test suite with:
- Test files
- Test data/fixtures
- Coverage report
- Bug reports (if found)
"""
```

Reviewer Agent

```
reviewer_agent_prompt = """You are a Code Review Agent.
Your expertise:
- Code quality, best practices
- Security vulnerabilities
- Performance bottlenecks
- Architecture review

Task: Review the following code
Code: {code}
Context: {context}
Output: Review report with:
- Issues found (severity: high/medium/low)
- Suggestions for improvement
- Security concerns
- Performance notes
"""
```

---

3. Tools Delegation

Setiap agent bisa punya akses ke tools berbeda. Orchestrator manage tool assignment.

```
# Tool definitions
tools = {
    "web_search": WebSearchTool(),
    "file_ops": FileOperationsTool(),
    "code_execution": CodeExecutionTool(),
    "terminal": TerminalTool(),
    "git": GitTool(),
    "database": DatabaseTool(),
}

# Agent-tool mapping
```

```
agent_tools = {
    "orchestrator": ["web_search", "file_ops"],
    "frontend": ["file_ops", "terminal"],
    "backend": ["file_ops", "terminal", "database"],
    "qa": ["code_execution", "file_ops"],
    "reviewer": ["file_ops", "terminal"],
}
```

Tool Types

Tool	Purpose	Access Control
Web Search	Research, documentation lookup	Orchestrator only
File Ops	Read/write files	All agents
Code Execution	Run tests, lint, build	QA, Reviewer
Terminal	Install packages, run commands	Dev agents
Git	Commit, branch, PR	Orchestrator
Database	Query, migrate	Backend only

---

4. Context Isolation

Penting untuk maintain **context boundaries** antar agents.

Per-Agent Context

Setiap worker hanya terima context yang relevan:

```
# BAD — semua context ke semua agent
orchestrator.broadcast(full_context)

# GOOD — filtered context per agent
orchestrator.delegate("frontend", {
    "task": "Build login page",
    "context": {
        "api_spec": api_spec,
        "design_tokens": design_tokens,
        "user_flow": "login → dashboard",
    }
})

orchestrator.delegate("backend", {
    "task": "Build auth API",
    "context": {
```

```
        "database_schema": db_schema,
        "auth_provider": "JWT",
        "security_requirements": reqs,
    }
})
```

Shared Knowledge

Beberapa context perlu di-share:

```
shared_knowledge = {
    "project_root": "/projects/my-app",
    "coding_standards": "PEP8, TypeScript strict",
    "git_branch": "feature/auth-system",
    "main_language": "Python 3.11",
}
```

Context Isolation Rules

1. **Need-to-know basis** — worker hanya terima context yang diperlukan
2. **No cross-agent communication** — worker komunikasi cuma via orchestrator
3. **Shared knowledge is read-only** — worker baca shared context, jangan ubah
4. **Result is owned** — worker output = milik worker, orchestrator yang aggregate

---

5. Parallel vs Sequential

Parallel Execution

Gunakan ketika tasks independent:

```
async def execute_parallel():
    """Frontend + Backend bisa parallel karena independent tasks"""
    tasks = [
        delegate("frontend", {"task": "Build UI components"}),
        delegate("backend", {"task": "Build API endpoints"}),
    ]
    results = await asyncio.gather(*tasks)
    return results
```

Sequential Execution

Gunakan ketika ada dependency:

```python
async def execute_sequential():
    """Backend dulu, baru QA karena QA butuh backend code"""
    backend_result = await delegate("backend", {"task": "Build API"})
    qa_result = await delegate("qa", {
        "task": "Test API",
        "depends_on": backend_result
    })
    return [backend_result, qa_result]
```

Hybrid

```python
async def execute_hybrid():
    """Parallel frontend + backend, sequential QA setelah keduanya selesai"""
    # Stage 1: parallel
    fe_task = delegate("frontend", {"task": "UI"})
    be_task = delegate("backend", {"task": "API"})
    fe_result, be_result = await asyncio.gather(fe_task, be_task)

    # Stage 2: sequential
    qa_result = await delegate("qa", {
        "task": "Integration test",
        "context": {"frontend": fe_result, "backend": be_result}
    })

    # Stage 3: parallel review
    reviews = await asyncio.gather(
        delegate("reviewer", {"code": fe_result}),
        delegate("reviewer", {"code": be_result}),
        delegate("reviewer", {"code": qa_result}),
    )

    return {
        "frontend": fe_result,
        "backend": be_result,
        "qa": qa_result,
        "reviews": reviews,
    }
```

---

6. Master Script: Orchestrator Workflow

Full orchestrator implementation:

```python
#!/usr/bin/env python3
"""Orchestrator — manage multi-agent workflow for feature development."""

import asyncio
import json
from datetime import datetime
from typing import Dict, Any, List

class Orchestrator:
    def __init__(self, project_root: str, model: str = "claude-3.5-sonnet"):
        self.project_root = project_root
        self.model = model
        self.agents = {}
        self.results = {}
        self.shared_knowledge = {}

    def register_agent(self, name: str, role: str, tools: List[str]):
        self.agents[name] = {
            "role": role,
            "tools": tools,
            "prompt": self._load_role_prompt(role)
        }

    def set_shared_knowledge(self, knowledge: Dict[str, Any]):
        self.shared_knowledge = knowledge

    async def delegate(self, agent_name: str, task: Dict[str, Any]) -> Dict[str,
        """Delegate task to agent, return result."""
        if agent_name not in self.agents:
            raise ValueError(f"Agent {agent_name} not registered")

        print(f"[{datetime.now()}] Delegating to {agent_name}: {task['task']}")

        # Build agent prompt with context
        agent_prompt = self.agents[agent_name]["prompt"].format(
            task=task["task"],
            context=json.dumps(task.get("context", {}), indent=2),
            shared=json.dumps(self.shared_knowledge, indent=2),
            tools=", ".join(self.agents[agent_name]["tools"])
        )

        # Simulate agent execution (in real system, call LLM)
        result = await self._execute_agent(agent_name, agent_prompt)
        self.results[agent_name] = result
        return result
```

```python
async def _execute_agent(self, name: str, prompt: str) -> Dict[str, Any]:
    """Execute agent — real implementation would call LLM API."""
    # Placeholder: dalam implementasi nyata, ini call ke Claude/Codex/etc
    return {
        "agent": name,
        "status": "completed",
        "output": f"[{name}] Task completed at {datetime.now()}"
    }

def aggregate_results(self) -> Dict[str, Any]:
    """Gabung semua hasil worker."""
    return {
        "timestamp": datetime.now().isoformat(),
        "project": self.shared_knowledge.get("project_name"),
        "agents": list(self.agents.keys()),
        "results": self.results,
        "summary": self._generate_summary()
    }

def _generate_summary(self) -> str:
    """Generate ringkasan eksekusi."""
    completed = [k for k, v in self.results.items()
                 if v.get("status") == "completed"]
    return f"{len(completed)}/{len(self.agents)} agents completed"

async def run_feature_workflow(self, feature_spec: Dict[str, Any]) -> Dict[s
    """Run complete feature development workflow."""
    print(f"=== Starting feature: {feature_spec['name']} ===")

    # Register default agents
    self.register_agent("frontend", "frontend", ["file_ops", "terminal"])
    self.register_agent("backend", "backend", ["file_ops", "terminal", "data
    self.register_agent("qa", "qa", ["code_execution", "file_ops"])
    self.register_agent("reviewer", "reviewer", ["file_ops", "terminal"])

    # Stage 1: Parallel FE + BE
    fe_task = self.delegate("frontend", {
        "task": feature_spec.get("frontend_task"),
        "context": {"spec": feature_spec}
    })
    be_task = self.delegate("backend", {
        "task": feature_spec.get("backend_task"),
        "context": {"spec": feature_spec}
    })
    fe_result, be_result = await asyncio.gather(fe_task, be_task)
```

```
        # Stage 2: Sequential QA
        qa_result = await self.delegate("qa", {
            "task": f"Test feature: {feature_spec['name']}",
            "context": {"frontend": fe_result, "backend": be_result}
        })

        # Stage 3: Review
        review_results = await asyncio.gather(
            self.delegate("reviewer", {"task": "Review FE", "context": fe_result
            self.delegate("reviewer", {"task": "Review BE", "context": be_result
            self.delegate("reviewer", {"task": "Review tests", "context": qa_res
        )

        return self.aggregate_results()


# ===== Usage =====
async def main():
    orchestrator = Orchestrator(project_root="/projects/my-app")

    feature = {
        "name": "User Authentication",
        "frontend_task": "Build login/register page with form validation",
        "backend_task": "Build auth API with JWT + refresh tokens",
    }

    result = await orchestrator.run_feature_workflow(feature)
    print(json.dumps(result, indent=2))

if __name__ == "__main__":
    asyncio.run(main())
```

---

7. Error Handling & Recovery

Worker Error

```
async def delegate_with_retry(self, agent_name: str, task: Dict[str, Any],
                              max_retries: int = 3) -> Dict[str, Any]:
    """Delegate dengan retry logic."""
    for attempt in range(max_retries):
        try:
            return await self.delegate(agent_name, task)
        except Exception as e:
            print(f"[{agent_name}] Attempt {attempt + 1} failed: {e}")
```

```
            if attempt == max_retries - 1:
                return {"status": "failed", "error": str(e)}
```

Orchestrator Recovery

```
async def recover_failed_agent(self, failed_agent: str, result: Dict[str, Any]):
    """Recovery strategy: reassign atau fallback."""
    print(f"Recovering {failed_agent}...")

    # Option 1: Reassign to another agent with same role
    if failed_agent == "frontend":
        return await self.delegate("backend", {
            "task": "Handle FE failure",
            "context": result
        })

    # Option 2: Simplify task
    return await self.delegate("qa", {
        "task": "Validate partial output",
        "context": result
    })
```

---

8. Latihan

Latihan 1: Build Orchestrator + 3 Workers

Buat orchestrator workflow sederhana (Python) dengan:

1. **Orchestrator** — manage task breakdown + aggregation
2. **Worker 1: Generator** — generate code dari spec
3. **Worker 2: Reviewer** — review generated code
4. **Worker 3: Tester** — generate + run tests

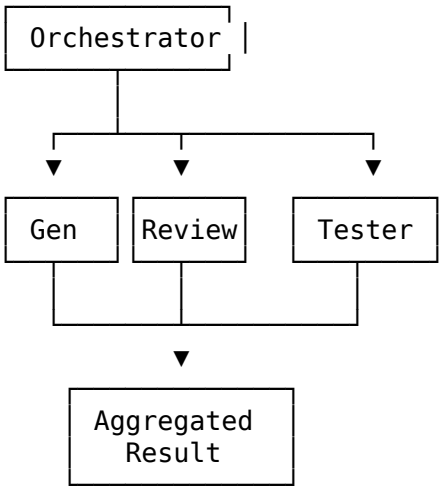
Spec:

```
spec = {
    "name": "Calculator API",
    "endpoints": [
        {"method": "POST", "path": "/add", "params": ["a", "b"]},
        {"method": "POST", "path": "/subtract", "params": ["a", "b"]},
        {"method": "POST", "path": "/multiply", "params": ["a", "b"]},
        {"method": "POST", "path": "/divide", "params": ["a", "b"]},
    ],
    "framework": "FastAPI",
    "language": "Python 3.11",
}
```

Workflow: 1. Generator bikin kode FastAPI dari spec 2. Reviewer review kode yang digenerate (security, style, correctness) 3. Tester bikin test untuk kode + run test 4. Orchestrator aggregate semua hasil

Latihan 2: Flow Diagram

Buat ASCII flow diagram untuk workflow yang kamu buat. Contoh:



Delivery

- Simpan orchestrator script di `53-ai-coding-agents/labs/orchestrator.py`
- Flow diagram di `53-ai-coding-agents/labs/flow-diagram.txt`
- Test dengan spec calculator API

Key Takeaways

1. **Orchestrator + Worker** adalah pattern fundamental multi-agent
2. **Role-based agents** — assign expertise, bukan generic
3. **Tools delegation** — setiap agent punya tool akses berbeda
4. **Context isolation** — need-to-know basis, jangan overload worker
5. **Parallel vs sequential** — bedakan independent vs dependent tasks
6. **Error handling** — retry, recovery, fallback strategy

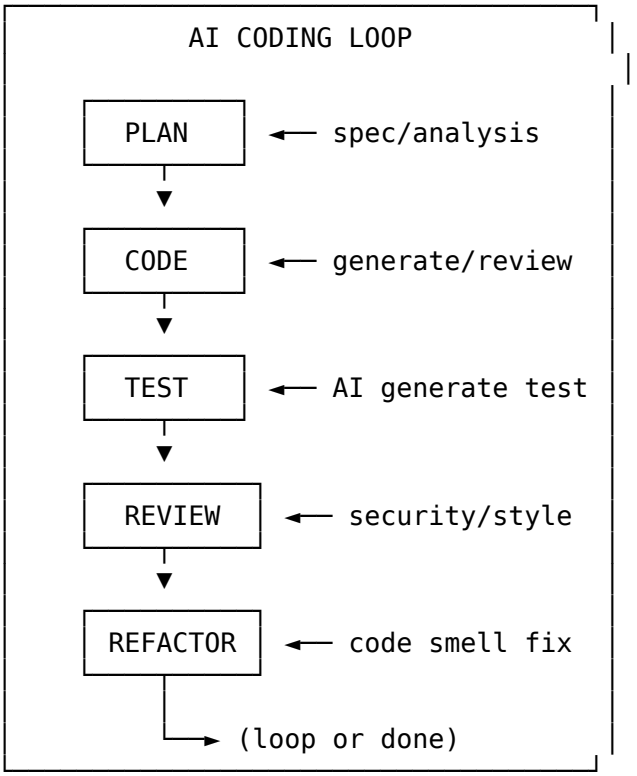
7. **Result aggregation** — orchestrator yang gabung, bukan worker

Session 03: AI Coding Loop

Level: Advanced
Duration: 2–3 hours
Objective: Kuasai AI Coding Loop — plan → code → test → review → refactor. Efisiensi loop, skip redundant steps, combine where possible.

---

1. The AI Coding Loop



Setiap iterasi makin mendekati production-ready code.

---

2. Plan: Spec → Analysis → Task Breakdown

Input: Product Spec / Feature Request

```
## Feature: File Upload with Preview

User can upload image files (jpg, png, webp) up to 5MB.
System generates thumbnail preview (200x200).
Supported files: images only.
Storage: local filesystem (for now).
```

AI Analysis

Prompt AI untuk breakdown:

```
Analyze this spec and produce:
1. Acceptance criteria (list of pass/fail conditions)
2. Technical tasks breakdown
3. Dependencies and risks
4. Estimated complexity (low/medium/high)

Spec: {spec}
```

Output: Task Breakdown

```
## Acceptance Criteria
- [ ] User can select file via button or drag-drop
- [ ] Only image files accepted (jpg, png, webp)
- [ ] File size limited to 5MB
- [ ] Thumbnail preview shown after upload
- [ ] Error shown for invalid files
- [ ] Progress indicator during upload

## Technical Tasks
1. Create FileUpload component (React/TS)
2. Create file validation utility
3. Create API endpoint POST /api/upload
4. Create thumbnail generation service (Pillow)
5. Create preview component
6. Add error handling + loading states

## Dependencies
- Pillow library for thumbnail generation
- File size validation on client + server
- CORS if frontend separate from backend
```

```
## Complexity
Overall: Medium
Frontend: Low
Backend: Medium
```

Template: Plan Prompt

```
Task: {feature_spec}

Sebagai AI Coding Agent, buat:

1. **Acceptance Criteria** — minimal 5 criteria, pass/fail format
2. **Task Breakdown** — frontend + backend tasks, dependency graph
3. **Risk Analysis** — potential issues + mitigations
4. **Technical Decisions** — framework choice, library, architecture

Format: Markdown checklist dengan code blocks untuk code structure.
JANGAN generate implementasi — cukup plan.
```

---

3. Code: Generate → Review → Refine

Generate with AI

```python
# Prompt untuk generate
prompt = f"""
Implementation task: {task}
Acceptance criteria: {criteria}
Tech stack: {tech_stack}
Existing code structure: {existing_code}

Generate implementation that:
1. Meets all acceptance criteria
2. Follows {project_standards} conventions
3. Includes error handling
4. Is production-ready

Return complete file contents with file paths.
"""
```

Iterative Refinement

```python
# Iteration 1: First generation
code_v1 = generate_code(prompt)
```

```
# Iteration 2: Review and refine
review_feedback = review_code(code_v1)
code_v2 = refine_code(code_v1, review_feedback)

# Iteration 3: Edge cases
edge_cases = analyze_edge_cases(code_v2)
code_v3 = refine_code(code_v2, edge_cases)
```

Prompt Iteration Strategy

Iteration	Focus	Example Prompt
v1	Happy path	“Generate the main implementation”
v2	Error handling	“Add error handling for edge cases”
v3	Performance	“Optimize for large inputs”
v4	Security	“Add security validation”
v5	Polish	“Clean up, add comments, type hints”

Template: Code Generation Prompt

```
Task: {specific_task}
Tech stack: {language, framework, libraries}
Acceptance criteria:
{criteria}

Constraints:
- {constraint_1}
- {constraint_2}

Generate complete implementation with:
1. All required files
2. Error handling
3. Type hints (if applicable)
4. Docstrings/comments for complex logic
5. Usage example
```

---

4. Test: AI Generate Test = Test Prompt

TDD with AI

```
RED (tulis test dulu) → GREEN (AI generate code) → REFACTOR

1. RED: AI generate test dari spec (sebelum ada code)
```

```
2. GREEN: AI generate code yang pass test
3. REFACTOR: AI refactor code + pastikan test masih pass
```

Test Prompt Template

```
Generate comprehensive tests for the following code:

```python
{pasted_code}
```

Requirements: - Cover all acceptance criteria - Include edge cases (empty input, null, overflow, etc.) - Include error cases (invalid input, permission, network error) - Achieve >80% code coverage

For each test provide: 1. Test name 2. Description of scenario being tested 3. Test code 4. Expected result

Format: pytest style, organized in classes by feature.

Test Quality Checklist

Aspect	Check
Covers acceptance criteria	Every AC has at least 1 test
Edge cases	Empty, null, boundary values
Error cases	Invalid input, timeout, auth failure
Coverage	Line coverage >80%
Readability	Clear test names, descriptive assertions
Isolation	No test depends on another test

Running Tests in Loop

```python
import subprocess
import json

def run_tests_and_report():
    """Run pytest and return structured results."""
    result = subprocess.run(
        ["pytest", "--json-report", "--cov=.", "-v"],
        capture_output=True, text=True
    )

    report = json.loads(result.stdout)
   failures = [t for t in report["tests"] if t["outcome"] == "failed"]
```
```

```
    return {
        "passed": report["summary"]["passed"],
        "failed": report["summary"]["failed"],
     "coverage": report.get("coverage", {}).get("total", 0),
        "failures": failures,
        "duration": report["summary"]["duration"]
    }
```

---

5. Review: AI Code Review Checklist

Security Scan

```
security_checklist = """
[ ] SQL injection — parameterized queries?
[ ] XSS — output escaped?
[ ] CSRF — tokens present?
[ ] Auth — correct permission checks?
[ ] Data validation — input sanitized?
[ ] Secrets — hardcoded passwords/tokens?
[ ] File upload — path traversal safe?
[ ] Rate limiting — brute force protection?
"""
```

Style Check

```
style_checklist = """
[ ] Follows language conventions (PEP8, ESLint, etc.)
[ ] Consistent naming (snake_case/camelCase)
[ ] No dead code / commented-out code
[ ] Proper indentation
[ ] Line length within limits
[ ] Imports sorted and clean
[ ] Function/method length reasonable (<50 lines)
"""
```

Architecture Check

```
architecture_checklist = """
[ ] Single Responsibility Principle
[ ] DRY (Don't Repeat Yourself)
[ ] Proper separation of concerns
[ ] Error handling at appropriate level
[ ] Logging for debugging
[ ] Configuration externalized
```

```
[ ] API design consistent (RESTful)
[ ] Database queries optimized
"""
```

AI Review Prompt

```
Review the following code changes:

```diff
{diff_content}
```

Evaluate against: 1. Security vulnerabilities (HIGH priority) 2. Code style and conventions 3. Architecture and design patterns 4. Performance implications 5. Potential bugs

For each issue found: - Severity: HIGH / MEDIUM / LOW - Location: file:line - Description - Suggested fix

Return as structured JSON:
```

```
{
  "issues": [
    {"severity": "HIGH", "file": "src/main.py", "line": 42,
     "description": "...", "suggestion": "..."}
  ],
  "summary": {"high": 2, "medium": 3, "low": 1},
  "approved": false
}
```

```
---

## 6. Refactor: AI Identify Code Smell

### Common Code Smells AI Can Detect

| Smell | AI Prompt |
|-------|-----------|
| **Long function** | "Identify functions >30 lines that can be split" |
| **Duplicate code** | "Find duplicate code blocks >5 lines" |
| **Magic numbers** | "Find hardcoded numeric values" |
| **Deep nesting** | "Find blocks with >3 levels of nesting" |
| **Large class** | "Identify classes with >10 methods or >300 lines" |
| **God object** | "Find classes that do too many different things" |

### Refactor Prompt

Analyze this code for refactoring opportunities:
```

{code}

For each opportunity: 1. Type of code smell 2. Current code location (file:line) 3. Refactored code suggestion 4. Expected benefit (readability, performance, maintainability)

Prioritize by impact: - HIGH: affects correctness or performance - MEDIUM: improves maintainability - LOW: minor style improvement

Return refactored version of the full file.

Example: AI-Initiated Refactor

```python
# BEFORE — AI identifies: repeated try/except pattern
def read_user(id):
    try:
        return db.query(User).get(id)
    except Exception as e:
        logger.error(f"Failed to read user {id}: {e}")
        raise

def read_product(id):
    try:
        return db.query(Product).get(id)
    except Exception as e:
        logger.error(f"Failed to read product {id}: {e}")
        raise

# AFTER — AI suggests decorator pattern
def db_operation(logger):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            try:
                return func(*args, **kwargs)
            except Exception as e:
                logger.error(f"DB operation failed: {e}")
                raise
        return wrapper
    return decorator

@db_operation(logger)
def read_user(id):
    return db.query(User).get(id)

@db_operation(logger)
```

```
def read_product(id):
    return db.query(Product).get(id)
```

---

7. Loop Efficiency

When to Skip Steps

Scenario	Skip	Reason
Simple function	Test + Review	Overhead > benefit
Boilerplate code	Plan	Clear from context
Prototype/PoC	Refactor	Not production-bound
Bug fix	Plan	Root cause already known
Config change	Code + Test + Review + Refactor	Just change config directly

When to Combine Steps

Combination	When
Code + Test	Simple module — generate code and test together
Review + Refactor	Small changes — review findings langsung di-refactor
Plan + Code	Well-understood task — plan implicit in code prompt
Test + Review	Test coverage review + code review simultaneous

Full Loop Time Estimates

Complexity	Plan	Code	Test	Review	Refactor	Total
Simple	2 min	3 min	2 min	1 min	—	~8 min
Medium	5 min	10 min	5 min	3 min	3 min	~26 min

Complexity	Plan	Code	Test	Review	Refactor	Total
Complex	15 min	30 min	15 min	10 min	10 min	~80 min

Loop Efficiency Tips

1. **Stop when "good enough"** — perfect is enemy of done
2. **Fail fast** — skip review if tests fail, langsung fix
3. **Batch review** — review multiple files at once
4. **Test isolation** — run only affected tests, not full suite
5. **Use AI for what AI is good at** — generation + review, not architecture
6. **Human in the loop** — important decisions (architecture, security) need human

---

8. Complete Loop Example

```python
#!/usr/bin/env python3
"""AI Coding Loop — full implementation."""

import subprocess
import json
from typing import Dict, Any

class AICodingLoop:
    def __init__(self, ai_model="claude"):
        self.model = ai_model
        self.iteration = 0
        self.results = []

    def plan(self, spec: str) -> Dict[str, Any]:
        """Phase 1: Plan — breakdown spec into tasks."""
        prompt = f"Breakdown this spec into acceptance criteria + tasks:\n\n{spe
        # In real implementation: call LLM API
        return {"tasks": ["task_1", "task_2"], "criteria": ["c1", "c2"]}

    def code(self, task: str, criteria: list) -> str:
        """Phase 2: Code — generate implementation."""
        self.iteration += 1
        prompt = f"Task: {task}\nCriteria: {criteria}\nGenerate implementation."
        # In real implementation: call LLM API
        return f"# Generated code for: {task}"
```

```
    def test(self, code: str) -> Dict[str, Any]:
        """Phase 3: Test — generate and run tests."""
        # Generate test
        test_prompt = f"Generate pytest for:\n{code}"
        # Run test
        result = subprocess.run(
            ["pytest", "-v", "--tb=short"],
            capture_output=True, text=True
        )
        return {
            "passed": result.returncode == 0,
            "output": result.stdout,
            "errors": result.stderr
        }

    def review(self, code: str) -> Dict[str, Any]:
        """Phase 4: Review — security + style check."""
        prompt = f"Review code:\n{code}"
        return {
            "issues": [],
            "approved": True,
            "suggestions": []
        }

    def refactor(self, code: str, suggestions: list) -> str:
        """Phase 5: Refactor — apply improvements."""
        prompt = f"Refactor this code:\n{code}\nSuggestions:\n{suggestions}"
        return f"# Refactored: {code}"

    def run_loop(self, spec: str, max_iterations: int = 3) -> Dict[str, Any]:
        """Run AI Coding Loop for specified iterations."""
        print(f"=== AI Coding Loop: Iteration {self.iteration + 1} ===")

        # Plan
        plan_result = self.plan(spec)
        print(f"[Plan] {len(plan_result['tasks'])} tasks identified")

        all_results = []
        for i in range(max_iterations):
            print(f"\n--- Iteration {i + 1} ---")

            for task in plan_result['tasks']:
                # Code
                code = self.code(task, plan_result['criteria'])
                print(f"[Code] {task}: {len(code)} chars generated")
```

```
                # Test
                test_result = self.test(code)
                print(f"[Test] Passed: {test_result['passed']}")

                if not test_result['passed']:
                    print("[Test] FAILED — fixing...")
                    code = self.code(f"Fix: {test_result['errors']}",
                                     plan_result['criteria'])
                    continue

                # Review
                review_result = self.review(code)
                print(f"[Review] Issues: {len(review_result['issues'])}")

                # Refactor
                code = self.refactor(code, review_result['suggestions'])
                print(f"[Refactor] Applied {len(review_result['suggestions'])} s

                all_results.append({
                    "iteration": i,
                    "task": task,
                    "code": code,
                    "tests": test_result,
                    "review": review_result
                })

        return {"iterations": max_iterations, "results": all_results}


# ===== Usage =====
if __name__ == "__main__":
    loop = AICodingLoop()
    spec = """
    Feature: User registration API
    - POST /api/register with email + password
    - Email format validation
    - Password min 8 chars
    - Return JWT token
    """
    result = loop.run_loop(spec, max_iterations=3)
    print(json.dumps(result, indent=2))
```

---

9. Latihan

Latihan: 3 Loop Iteration on a Feature

Pilih satu fitur berikut:

Option A: To-Do List API

```
- GET /todos — list semua todos
- POST /todos — create todo (title, priority)
- PATCH /todos/:id — update status
- DELETE /todos/:id — delete todo
```

Option B: URL Shortener

```
- POST /shorten — create short URL
- GET /:code — redirect to original
- GET /:code/stats — view click count
```

Option C: (kamu tentukan sendiri)

Iterasi 1:

1. **Plan** — acceptance criteria + task breakdown (file: `PLAN.md`)
2. **Code** — generate implementasi lengkap (1 file atau lebih)
3. **Test** — generate test untuk kode iterasi 1
4. **Review** — review kode iterasi 1, catat issues

Iterasi 2:

1. Fix issues dari review iterasi 1
2. Add error handling + edge cases
3. Update test
4. Review lagi

Iterasi 3:

1. Add security (input validation, rate limiting, etc.)
2. Add logging
3. Refactor code smells
4. Final review

Output per Iterasi

Simpan di folder `53-ai-coding-agents/labs/loop/`:

```
labs/loop/
├── iteration-01/
│   ├── code.py           # Generated code
```

```
│   ├── test_code.py      # Generated tests
│   ├── review.md         # Review report
│   └── PLAN.md           # Plan document
├── iteration-02/
│   ├── code.py
│   ├── test_code.py
│   └── review.md
└── iteration-03/
    ├── code.py
    ├── test_code.py
    └── review.md
```

Checklist Delivery

- ☐ Plan document lengkap dengan acceptance criteria
- ☐ Code generation untuk setiap iterasi
- ☐ Test generation (minimal 5 test per iterasi)
- ☐ Review report dengan issues + suggestions
- ☐ Refactor di iterasi 3
- ☐ Dokumentasi perbedaan antar iterasi

Key Takeaways

1. **Loop is not optional** — even AI-generated code needs plan → test → review
2. **Start small** — iterasi 1: happy path only, iterasi 2: edge cases, iterasi 3: polish
3. **Test first (TDD)** — AI generate test from spec before code
4. **Review security always** — AI can miss security vulnerabilities
5. **Know when to stop** — not every task needs full loop
6. **Combine steps** — reduce overhead for simple tasks
7. **Document iterations** — track improvement across loops

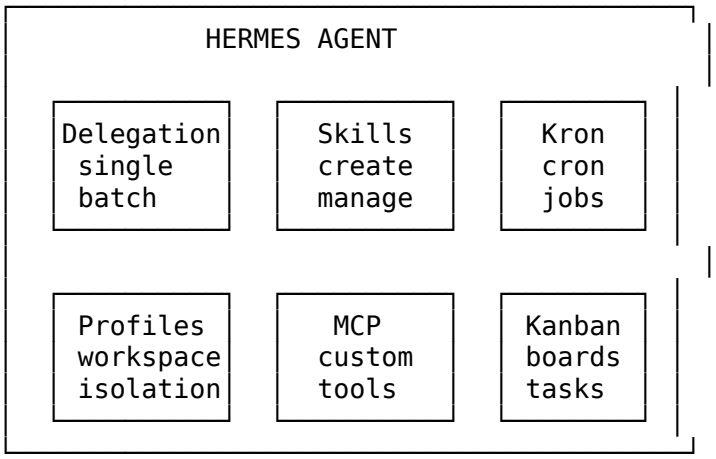
Session 04: Hermes Agent Mastery

Level: Advanced
Duration: 2–3 hours
Objective: Kuasai Hermes Agent — delegation, skills, kron, profiles, MCP, kanban. Jadi power user yang bisa build workflow kompleks.

---

1. Features Overview

Hermes Agent adalah **agent orchestrator** — bukan cuma AI chat. Feature set-nya dirancang untuk multi-agent workflow automation.



---

2. Delegation

Delegation adalah cara Hermes Agent men-turunkan task ke sub-agent.

Single Delegation

```
# Delegate task langsung ke Hermes
hermes "Build a FastAPI CRUD for users"

# Hermes akan:
# 1. Breakdown task
# 2. Execute subtasks (read/write files, run commands)
# 3. Report result
```

Di dalam session Hermes, delegasi terjadi otomatis ketika task membutuhkan:

- **File operations** → sub-agent file handler
- **Code execution** → sub-agent terminal executor
- **Web search** → sub-agent web search (Firecrawl)
- **Image generation** → sub-agent FAL
- **Browser automation** → sub-agent browser use

Batch Delegation

```python
# Dalam skill atau script, kamu bisa batch delegate
from hermes import Delegation

results = Delegation.batch([
    {"task": "Check API health", "timeout": 30},
    {"task": "Run database migration", "timeout": 120},
    {"task": "Deploy to staging", "timeout": 300},
])
```

Delegation Flow

```
User Task
    │
    ▼
Hermes Agent (Orchestrator)
    │
    ├── Task Analysis → breakdown → priority
    │
    ├── Delegate to Sub-Agent 1: [tool: file_ops]
    │   └── Read/modify files
    │
    ├── Delegate to Sub-Agent 2: [tool: terminal]
    │   └── Execute commands
    │
    ├── Delegate to Sub-Agent 3: [tool: web_search]
    │   └── Fetch documentation
    │
    └── Aggregate → verify → report to user
```

Delegation Tips

Tip	Detail
Be specific	“Create file auth.py with login function” > “Buat auth”
One task at a time	Complex task: break into multiple delegations
Set expectations	“Run test, if fail, report error and stop”
Use context	Provide file paths, function names, examples

---

3. Kron Jobs

Kron = cron job system di Hermes Agent. Untuk menjadwalkan task periodic.

Create Kron Job

```
# Via hermes CLI
hermes kron create "Daily backup" "0 2 * * *" "Run backup script"
hermes kron create "Weekly report" "0 9 * * 1" "Generate weekly metrics"
hermes kron list
```

Kron Job Format

```
# ~/.hermes/kron/daily-backup.yaml
name: Daily Backup
schedule: "0 2 * * *"      # Every day at 2 AM
task: |
  1. Run database dump
  2. Compress backup files
  3. Upload to S3
  4. Clean up old backups (retain 7 days)
notify: true                # Notify on completion
timeout: 600                # Max 10 minutes
retry: 3                    # Retry 3 times on failure
```

Kron Management Commands

```
hermes kron list                    # List all jobs
hermes kron show daily-backup       # Show job details
hermes kron pause daily-backup      # Pause without deleting
hermes kron resume daily-backup     # Resume paused job
hermes kron delete daily-backup     # Remove job
hermes kron logs daily-backup       # View execution logs
```

Kron Use Cases

Use Case	Schedule	Task
Daily backup	`0 2 * * *`	Database + file backup
Code cleanup	`0 0 * * 0`	Remove temp files, prune branches
Health check	`*/30 * * * *`	Ping services, report status
Weekly report	`0 9 * * 1`	Generate metrics, send email
Dependency update	`0 8 * * 1`	Check + update outdated packages

4. Skills

Skills adalah **reusable workflow** — kumpulan instruksi yang bisa kamu simpan, version, dan panggil ulang.

Create Skill

```
# Via CLI
hermes skill create my-workflow

# Ini akan buat file di ~/.hermes/skills/my-workflow/
```

Skill Structure

```yaml
# ~/.hermes/skills/my-workflow/skill.yaml
name: my-workflow
version: 1.0.0
description: "Standard workflow for creating a new API endpoint"
author: "user"
tags: [api, fastapi, backend]

steps:
  - name: analyze-spec
    prompt: "Analyze this API spec and create task breakdown"
    tools: [web_search]

  - name: generate-code
    prompt: "Generate FastAPI implementation based on spec"
    tools: [file_ops, terminal]
    depends_on: [analyze-spec]

  - name: generate-tests
    prompt: "Generate pytest for the API"
    tools: [file_ops, code_execution]
    depends_on: [generate-code]

  - name: review-code
    prompt: "Review generated code for security + style"
    tools: [file_ops]
    depends_on: [generate-code]

triggers:
  - type: command
    pattern: "create endpoint *"
  - type: file_watch
    pattern: "specs/*.yaml"
```

Skill Versioning

```
hermes skill version my-workflow    # Show version history
hermes skill update my-workflow     # Update to new version
hermes skill rollback my-workflow 1 # Rollback to v1
```

Skill Management Commands

```
hermes skill list                   # List all skills
hermes skill show my-workflow       # Show skill details
hermes skill run my-workflow        # Run skill immediately
hermes skill delete my-workflow     # Remove skill
hermes skill export my-workflow     # Export as sharable file
hermes skill import ./skill.yaml    # Import skill from file
```

Skill Improvement

Setelah skill digunakan beberapa kali, review dan improve:

```yaml
# Improved version 1.1.0
version: 1.1.0
changelog:
  - "Added error handling step"
  - "Improved prompt specificity"
  - "Added async support"

steps:
  - name: analyze-spec
    prompt: "Analyze API spec. Identify: endpoints, params, auth, error codes"
    tools: [web_search]

  - name: generate-code
    prompt: |
      Generate FastAPI implementation with:
      - Input validation (Pydantic)
      - Error handling for all HTTP status codes
      - Async database queries
      - Proper HTTP exception handling
    tools: [file_ops, terminal]
    depends_on: [analyze-spec]
  # ... etc
```

---

5. Profiles

Profiles memungkinkan **workspace isolation**. Setiap profile punya skills, plugins, cron, dan memory sendiri.

Why Use Profiles?

Scenario	Without Profile	With Profile
Work on 2 projects	Skills campur aduk	Isolasi per project
Personal vs work	Context pollution	Clean separation
Different tech stacks	Prompt confusion	Tailored skills
Team collaboration	Override conflicts	Shared profile

Setup Profile

```
# List profiles
hermes profile list

# Create new profile
hermes profile create web-dev

# Switch profile
hermes profile use web-dev

# Current profile
hermes profile current
```

Profile Structure

```
~/.hermes/profiles/web-dev/
├── skills/               # Profile-specific skills
├── plugins/              # Profile-specific plugins
├── cron/                 # Profile-specific cron jobs
├── memories/             # Profile-specific memory
└── profile.yaml          # Profile config
```

Profile Config

```yaml
# ~/.hermes/profiles/web-dev/profile.yaml
name: web-dev
model: claude-3.5-sonnet
tools:
  - web_search
  - file_ops
```

```yaml
  - terminal
  - code_execution
environment:
  python_version: "3.11"
  node_version: "20"
  workdir: "/projects/web-dev"

skills:
  enabled:
    - fastapi-crud
    - react-component
    - test-generator
  disabled:
    - data-pipeline
    - devops-deploy
```

Per-Project Profile

```bash
# Setup profile for current project
cd /home/midory/rpl-ai-curriculum
hermes profile create rpl-curriculum
hermes profile use rpl-curriculum

# Profile will remember project context
```

Memory Isolation

```yaml
# ~/.hermes/profiles/web-dev/memories/context.yaml
project_name: "Web Dev Platform"
tech_stack: "FastAPI + React + PostgreSQL"
coding_standards: "PEP8, Prettier, ESLint"
team: ["alice", "bob", "charlie"]
repository: "git@github.com:org/web-platform.git"
```

---

6. MCP (Model Context Protocol)

MCP memungkinkan Hermes Agent terhubung dengan **external tools dan services**.

Add MCP Server

```bash
# Add MCP server
hermes mcp add my-server \
```

```
  --url https://mcp.my-server.com \
  --api-key $MCP_KEY

# List connected MCP servers
hermes mcp list

# Test connection
hermes mcp ping my-server
```

Custom Tools via MCP

```
# ~/.hermes/mcp/custom-tools.yaml
tools:
  - name: github_issue_create
    description: "Create GitHub issue"
    input:
      repo: string
      title: string
      body: string
      labels: string[]
    server: github-mcp

  - name: slack_notify
    description: "Send Slack notification"
    input:
      channel: string
      message: string
      priority: "low" | "normal" | "high"
    server: slack-mcp

  - name: deploy_service
    description: "Deploy service to Kubernetes"
    input:
      service: string
      namespace: string
      image_tag: string
    server: k8s-mcp
```

Webhook Triggers

```
# ~/.hermes/mcp/webhooks.yaml
webhooks:
  - name: github-push
    url: /webhooks/github
    events: ["push", "pull_request"]
```

```
    action: "Run tests on push"

  - name: deploy-complete
    url: /webhooks/deploy
    events: ["deploy.success", "deploy.failure"]
    action: "Notify team via slack"
```

MCP Use Cases

MCP Server	Tools Provided	Use Case
GitHub MCP	Issues, PR, commits, reviews	Dev workflow automation
Slack MCP	Messages, channels, threads	Team notifications
Jira MCP	Tickets, sprints, boards	Project management
Database MCP	SQL queries, migrations	Data operations
Kubernetes MCP	Deployments, pods, logs	Infrastructure management
Custom API MCP	Any REST/GraphQL API	Business-specific tools

---

7. Kanban

Kanban board untuk **task management multi-session**. Mirip Trello/Notion tapi di dalam Hermes.

Create Board

```
# Create new board
hermes kanban create "Module 53 Development"

# List boards
hermes kanban list
```

Manage Tasks

```
# Add task
hermes kanban add "Module 53" \
  --title "Write session 01" \
  --description "AI coding tools comparison" \
  --priority high \
  --due "2026-07-06"

# Move task
hermes kanban move "Write session 01" --to in-progress
```

```
hermes kanban move "Write session 01" --to done

# List board
hermes kanban show "Module 53"
```

Board Structure

```yaml
# ~/.hermes/kanban/module-53.yaml
board:
  name: "Module 53 Development"
  columns:
    - name: backlog
      tasks:
        - title: "Finalize session 04"
          priority: high
          status: open

    - name: in-progress
      tasks:
        - title: "Write session 03"
          priority: high
          status: active
          started: "2026-07-05"

    - name: review
      tasks:
        - title: "Write session 02"
          priority: medium
          status: review

    - name: done
      tasks:
        - title: "Write session 01"
          priority: high
          status: completed
          completed: "2026-07-05"
```

Multi-Session Tracking

```
# Continue task from previous session
hermes kanban continue "Write session 03"

# This will:
# 1. Check current session context
# 2. Load relevant files
```

```
# 3. Resume where left off
```

Kanban Commands

```
hermes kanban list                          # List all boards
hermes kanban create "Board Name"           # Create board
hermes kanban show "Board Name"             # Show board with tasks
hermes kanban add "Board" --title "T"       # Add task
hermes kanban move "Task" --to "column"     # Move task
hermes kanban done "Task"                   # Mark done
hermes kanban delete "Task"                 # Remove task
hermes kanban continue "Task"               # Resume task
hermes kanban archive "Board Name"          # Archive board
```

---

8. Tips: Prompting Hermes

Prompt Structure

```
[Context] + [Task] + [Constraints] + [Output Format]

Contoh baik:
"Di project /projects/my-api, buat file auth.py
dengan fungsi login(email, password) yang return JWT token.
Gunakan FastAPI + python-jose. Include error handling.
Output: complete file content."

Contoh kurang baik:
"Buat auth."
```

Context Management

```
# Cara inject context ke Hermes:

## 1. File context
"Baca /projects/my-api/README.md, lalu implementasi sesuai doc"

## 2. Code context
"Lihat fungsi get_user() di models.py, buat test untuk fungsi itu"

## 3. Session context
(dalam sesi yang sama, Hermes ingat konteks percakapan)

## 4. Skill context
"Gunakan skill fastapi-crud untuk buat endpoint products"
```

Tool Selection Tips

Tool	When to Use	Prompt Pattern
file_ops	Read/write files	"Buat file ... berisi ..."
terminal	Run commands	"Jalankan ..."
web_search	Research	"Cari dokumentasi tentang ..."
code_execution	Test code	"Jalankan test untuk ..."
browser_use	Web tasks	"Buka website ..., lakukan ..."

Power User Tips

1. **Multi-step prompting** — Break complex request into steps
2. **Use skills** — Jangan repeat workflow, simpan sebagai skill
3. **Context window** — File besar? Baca sebagian, bukan seluruhnya
4. **Iterative refinement** — "Better", "Add error handling", "Now optimize"
5. **File references** — Use absolute paths, jangan relatif ambiguous
6. **Be explicit about tools** — "Cari di web..." triggers web search
7. **Use profiles** — Switch profile per project untuk memory isolation

---

9. Latihan

Latihan 1: Create Skill from Workflow

1. Pilih workflow yang sering kamu lakukan (misal: setup FastAPI project, create React component, run test suite)
2. Catat langkah-langkahnya

Contoh workflow: "Setup FastAPI Project"

```yaml
name: setup-fastapi
version: 1.0.0
description: "Setup new FastAPI project with standard structure"

steps:
  - name: create-project-structure
    prompt: |
      Buat struktur folder FastAPI project:
      - src/ (main app)
      - src/api/ (routers)
      - src/models/ (database models)
      - src/schemas/ (pydantic schemas)
```

```
      - src/services/ (business logic)
      - tests/
      - alembic/ (migrations)

  - name: setup-dependencies
    prompt: |
      Buat requirements.txt dengan:
      fastapi, uvicorn, sqlalchemy, alembic,
      pydantic, python-jose, passlib, pytest

  - name: create-base-app
    prompt: |
      Buat src/main.py FastAPI app dengan:
      - CORS middleware
      - Health check endpoint
      - Router includes
      - Exception handlers
```

3. Simpan sebagai skill:

```
hermes skill create setup-fastapi
# Edit ~/.hermes/skills/setup-fastapi/skill.yaml
hermes skill run setup-fastapi
```

Latihan 2: Setup Profile + Kanban

1. Buat profile baru untuk salah satu project kamu:

```
hermes profile create my-project
hermes profile use my-project
```

2. Setup kanban board untuk project:

```
hermes kanban create "My Project"
hermes kanban add "My Project" \
  --title "Task 1" --priority high
hermes kanban add "My Project" \
  --title "Task 2" --priority medium
hermes kanban add "My Project" \
  --title "Task 3" --priority low
```

3. Practice move tasks:

```
hermes kanban move "Task 1" --to in-progress
hermes kanban show "My Project"
```

Latihan 3: MCP Integration

Jika punya akses ke MCP server:

1. Add MCP server:

```
hermes mcp add my-server --url <mcp-url>
hermes mcp list
```

2. Create custom tool yang menggunakan MCP
3. Test tool via Hermes

Delivery

- ☐ Skill file di `~/.hermes/skills/<nama-skill>/skill.yaml`
- ☐ Profile aktif untuk project
- ☐ Kanban board dengan minimal 3 tasks
- ☐ Screenshot/catatan dari latihan

Key Takeaways

1. **Delegation** — Hermes auto-delegate task ke sub-agent yang tepat
2. **Skills** — Simpan workflow reusable, version, improve
3. **Kron** — Jadwalkan periodic tasks dengan cron syntax
4. **Profiles** — Isolasi workspace per project, memory terpisah
5. **MCP** — Extend Hermes dengan custom tools dan webhooks
6. **Kanban** — Track tasks multi-session, resume kapan saja
7. **Power user** — Combine semua feature untuk workflow automation

---

Next Steps After Module 53

- Explore **MCP marketplace** untuk tools tambahan
- Build **complex skill chains** (skill yang memanggil skill lain)
- Setup **multi-profile workspace** untuk semua project
- Automate **CI/CD pipeline** dengan kron + MCP
- Build **custom MCP server** untuk business-specific tools